

System Design: Usage Metering & Billing Engine

Capstone Phase 1 Database Schema

Database Schema

The database models are designed to enforce tenant isolation, support idempotent metering, and mirror Stripe subscription states. Money is stored as integer cents to avoid floating-point errors.

1. plans

Defines the subscription tiers and their hard limits.

- **id**: UUID (Primary Key)
- **name**: VARCHAR (e.g., 'Free', 'Pro')
- **api_call_limit**: INTEGER
- **ai_token_limit**: INTEGER
- **base_price_cents**: INTEGER

2. tenants

Represents customer organizations. Customers belong to tenants, and data is isolated per tenant.

- **id**: UUID (Primary Key)
- **name**: VARCHAR
- **stripe_customer_id**: VARCHAR (Nullable)
- **created_at**: TIMESTAMP

3. subscriptions

Tracks the billing cycle and mirrors the payment truth from Stripe via webhooks.

- **id**: UUID (Primary Key)
- **tenant_id**: UUID (Foreign Key to **tenants**)
- **plan_id**: UUID (Foreign Key to **plans**)
- **stripe_subscription_id**: VARCHAR (Unique)
- **status**: VARCHAR (*active, past_due, canceled, unpaid, incomplete, expired*)
- **current_period_start**: TIMESTAMP
- **current_period_end**: TIMESTAMP

4. usage_events

An append-only ledger handling exactly-once metering.

- **id**: UUID (Primary Key)
- **tenant_id**: UUID (Foreign Key to **tenants**)
- **usage_type**: VARCHAR (*api_call, ai_token*)
- **quantity**: INTEGER
- **idempotency_key**: VARCHAR (**Unique constraint** with **tenant_id**)
- **metadata**: JSONB (Tracks specific token categories)
- **created_at**: TIMESTAMP

Key Design Constraints

- **Exactly-once metering**: A composite unique constraint on (**tenant_id**, **idempotency_key**) guarantees that retried network requests cannot duplicate usage charges.
- **Quota enforcement**: System boundaries check current usage between **current_period_start** and **current_period_end** before logging new events, throwing 429 or 402 HTTP status codes as needed.